IU · PROJECT: SOFTWARE ENGINEERING (CSEMCSPSE01) · PORTFOLIO PART 3

Tulsi Tiffins POS

Installation and Operating Manual

How to open the hosted application, install it on your own computer with one Docker command, run the tests, deploy it to the cloud – and how to use it once it runs.

Author	**Praveen Mysore Jagannath** · 10251728
Tutor	Niknejad, Naghmeh
Document version	**1.0.0** (matches software release v1.0.0)
Date	28 October 2026
Source code	github.com/mjpraveen04/tulsi-pos
Live application	https://tulsi-web.onrender.com

Contents

1 Quick reference

2 Option A – use the hosted application

3 Option B – install locally with Docker Compose

4 Configuration

5 Running the automated tests

6 Developer mode without Docker

7 Cloud deployment (Render + Neon)

8 Using the application – first steps

9 Troubleshooting

10 Stopping, resetting and uninstalling

VERIFIED ON 28 OCTOBER 2026

Every command in Sections 3, 5 and 6 was run on macOS 26 with Docker Desktop on that date. Section 3 was run against a **fresh clone from GitHub** (commit 90c36c1) in an empty folder, built with `--no-cache` and started on an empty database: images built in 38 s (base images already downloaded), the API was ready about 5 s after start, and a cashier sale returned bill 1042 for ₹84.00 with ₹416.00 change. The hosted application in Section 2 was smoke-tested on the same day.

1 Quick reference

What	Value
Hosted application	**https://tulsi-web.onrender.com**
Source code	https://github.com/mjpraveen04/tulsi-pos (branch `main`, tag `v1.0.0`)
Local install	`docker compose up --build` in the repository folder, then http://localhost:5173
API documentation (local)	http://localhost:8000/docs (Swagger UI, generated from the code)
Run the tests	Section 5 – 120 tests against PostgreSQL, about 2 minutes

1.1 Login data

The same three demo accounts exist in the hosted application and in every local installation; they are created automatically on first start. On the login screen, tap a name, then enter the four-digit PIN.

Name	Role	PIN	Can use
Anita R.	Manager	`1234`	Register, Inventory, Orders, Staff
Ravi K.	Cashier	`1111`	Register, own bill history
Priya M.	Cashier	`2222`	Register, own bill history

> **Before real use**, change or deactivate these demo accounts and set a strong `JWT_SECRET` (Section 4). The demo PINs are public in this manual and in the repository.

2 Option A – use the hosted application

No installation is needed. Open **https://tulsi-web.onrender.com** in a current browser (Chrome, Edge, Firefox or Safari) on a laptop or tablet and sign in with one of the accounts in Section 1.1.

> **The first request can take 30–60 seconds.** The application runs on free hosting plans: the API goes to sleep after 15 minutes without traffic and is woken by the next request. If the login screen shows no names or a loading message, wait a minute and reload the page. After that, the application responds normally.

The hosted instance runs exactly the code of the `main` branch: Render rebuilds it automatically on every push. It uses its own PostgreSQL database on Neon, so sales and stock changes made there are stored permanently and are visible to everyone who uses the demo. Section 7 describes how this deployment is set up.

3 Option B – install locally with Docker Compose

The whole system – database, API and web front end – runs as three containers started with one command (NFR-14). Nothing else needs to be installed on the computer.

3.1 Prerequisites

Requirement	Details
Docker with Compose v2	Docker Desktop 4.x (Windows, macOS) or Docker Engine 24+ with the Compose plugin (Linux). Check with `docker compose version`.
Memory and disk	About 2 GB free RAM for Docker and 1.5 GB disk for the images.
Free ports	5173 (web), 8000 (API), 5432 (database). Other ports can be chosen (Section 3.4).
Internet access	Only for the first build, to download the base images and packages.
Git (optional)	To clone the repository. Alternatively use the code zip from the submission or GitHub's *Download ZIP*.

3.2 Steps

1. **Get the code.**

   ```
   git clone https://github.com/mjpraveen04/tulsi-pos.git
   cd tulsi-pos
   ```

   or unzip `MysoreJagannath-Praveen_10251728_PSE_Submission_Code.zip` and open a terminal in the unzipped folder.

2. **Start Docker Desktop** and wait until it reports that it is running.

3. **Optional – create a configuration file.** Every setting has a safe default, so this step can be skipped.

   ```
   cp .env.example .env        # Windows PowerShell: copy .env.example .env
   ```

4. **Build and start** all three containers:

   ```
   docker compose up --build
   ```

   The first build downloads base images and takes a few minutes; later starts take seconds. The API log shows these three lines when it is ready:

   ```
   [entrypoint] applying Alembic migrations...
   [entrypoint] seeding demo data (idempotent)...
   [entrypoint] starting uvicorn...
   ```

5. **Open http://localhost:5173** and sign in, for example as Anita R. with PIN `1234`.

To run the stack in the background instead, use `docker compose up -d --build`; `docker compose logs -f api` then shows the API log.

3.3 What starts

Service	Image	Address on your computer	Role
`db`	postgres:16-alpine	localhost:5432	PostgreSQL database; data kept in the Docker volume `tulsi_db_data`
`api`	built from `backend/Dockerfile` (Python 3.11)	http://localhost:8000	FastAPI service. On every start it applies the Alembic migrations, adds the demo staff and menu if they are missing, then starts uvicorn

Service	Image	Address on your computer	Role
`web`	built from `frontend/Dockerfile` (Node 20 build, nginx 1.27)	http://localhost:5173	Serves the React application and forwards `/api` and `/media` to the API, so the browser talks to one address

3.4 Check that it works

```
curl http://localhost:5173/api/health          # {"status":"ok"}
curl http://localhost:5173/api/users/pin-targets # the three demo staff
curl -i http://localhost:5173/api/register/menu  # 401 — a token is required
```

In the browser: sign in as Ravi K. (`1111`), tap *Filter Coffee* twice, choose **Charge ₹84.00 → Cash**, enter `500` and confirm. The receipt shows bill 1042 and change of ₹416.00; the stock badge of the item drops by two.

3.5 If a port is already in use

Choose other host ports in `.env` or directly on the command line. The containers talk to each other inside Docker, so only the published ports change:

```
WEB_PORT=5180 API_PORT=8010 DB_PORT=5440 docker compose up --build
```

The application is then at http://localhost:5180. A local PostgreSQL installation is the most common cause of a conflict on port 5432.

4 Configuration

All settings come from environment variables. Docker Compose reads them from a `.env` file in the repository folder (template: `.env.example`); every variable has a default in `docker-compose.yml`. Nothing is configured in the code.

Variable	Default	Purpose
`DATABASE_URL`	`postgresql+psycopg2://tulsi:tulsi@db:5432/tulsi_pos`	Database connection used by the API. Host `db` is the Compose service name.
`POSTGRES_USER` / `_PASSWORD` / `_DB`	`tulsi` / `tulsi` / `tulsi_pos`	Credentials the database container is created with. If changed, change `DATABASE_URL` to match.
`JWT_SECRET`	`dev-secret-change-me`	Key that signs login tokens. **Set a long random value for any real use**, e.g. `openssl rand -hex 32`.
`JWT_EXPIRE_HOURS`	`8`	Lifetime of a login – one shift (NFR-08).
`UPI_ID` / `UPI_PAYEE_NAME`	`tulsitiffins@upi` / `Tulsi Tiffins`	Encoded in the UPI QR code shown at payment. Set the café's real UPI ID.
`GST_RATE`	`0.05`	GST applied to every bill.
`SEED_ON_START`	`true`	Adds the demo staff and menu if missing. Existing data is never overwritten.
`WEB_PORT` / `API_PORT` / `DB_PORT`	`5173` / `8000` / `5432`	Ports published on the host.

After changing `.env`, restart with `docker compose up -d`. Product photos are stored in the database, so no extra file storage needs to be configured or backed up.

5 Running the automated tests

The 120 backend tests run against a real PostgreSQL database, because the concurrency and rollback release gates need real row locks and a real `CHECK` constraint. The test setup creates a separate database `tulsi_pos_test` and migrates it with Alembic; the application's data is not touched.

Additional prerequisite: **Python 3.11** (3.12 also works).

```
docker compose up -d db                      # the database container is enough
cd backend
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt    # Windows: .venv\Scripts\pip ...
PATH="$PWD/.venv/bin:$PATH" .venv/bin/python -m pytest -q
```

Expected result: `120 passed` in about two minutes. Only the six release gates: add `-m gate` (about 4 s).

> **Two things that make every test fail with an error:** (1) the virtual environment is not on `PATH` – the test setup calls the `alembic` command and reports `FileNotFoundError: alembic`; (2) the database runs on another port than 5432. In that case pass its address: `TEST_DATABASE_URL="postgresql+psycopg2://tulsi:tulsi@localhost:5440/tulsi_pos_test"` before the command.

The front end has no unit tests (TD-02); its correctness against the API is checked by the TypeScript compiler. `npm run build` in `frontend/` type-checks every file first and fails on any mismatch with the API types.

6 Developer mode without Docker

For changing the code, the API and the front end can run directly on the computer with automatic reload. The database still runs in Docker. Additional prerequisites: Python 3.11 and Node.js 20 or newer.

```
# terminal 1 – database
docker compose up -d db

# terminal 2 – API on http://localhost:8000
cd backend
.venv/bin/alembic upgrade head
DATABASE_URL="postgresql+psycopg2://tulsi:tulsi@localhost:5432/tulsi_pos" \
  .venv/bin/uvicorn app.main:app --reload

# terminal 3 – front end on http://localhost:5173
cd frontend
npm install
npm run dev
```

The Vite development server forwards `/api` and `/media` to http://localhost:8000; set `VITE_API_TARGET` to use another address. Stop the Docker `web` and `api` containers first if they occupy ports 5173 and 8000. The `alembic` and `uvicorn` commands also need `DATABASE_URL` when the database is not on port 5432.

7 Cloud deployment (Render + Neon)

The hosted application is deployed from the file `render.yaml` (a Render *Blueprint*) in the repository. It defines two services built from the same Dockerfile and front end as the local installation; the database is a free PostgreSQL project on Neon, because Render's free database is deleted after 30 days.

Part	Provider and plan	Configuration
`tulsi-api`	Render web service, Docker, free	Built from `backend/Dockerfile`; health check `/api/health`; environment variables from `render.yaml`; `JWT_SECRET` generated by Render; `DATABASE_URL` entered by hand
`tulsi-web`	Render static site, free	`npm ci && npm run build`; publishes `dist/`; rewrites `/api/*` and `/media/*` to the API and all other paths to `index.html`
Database	Neon PostgreSQL 18, free, EU region	Connection string with `sslmode=require`

7.1 Deploying a new copy

1. Fork or push the repository to your own GitHub account.
2. At neon.tech create a project and copy the connection string. Change its start from `postgresql://` to `postgresql+psycopg2://` and keep `?sslmode=require` at the end.
3. At render.com choose **New → Blueprint**, connect the repository and paste the string as `DATABASE_URL` when asked. Render creates both services and starts the first build.
4. Render assigns the API a host name such as `tulsi-api-xxxx.onrender.com`. If it differs from the one in `render.yaml`, change the two rewrite destinations there and push the change.
5. Open the address of the `tulsi-web` service. Migrations and demo data are applied by the API on its first start.

Every later push to `main` redeploys both services automatically. Free-plan limits: the API sleeps after 15 minutes without traffic (first request 30–60 s).

8 Using the application – first steps

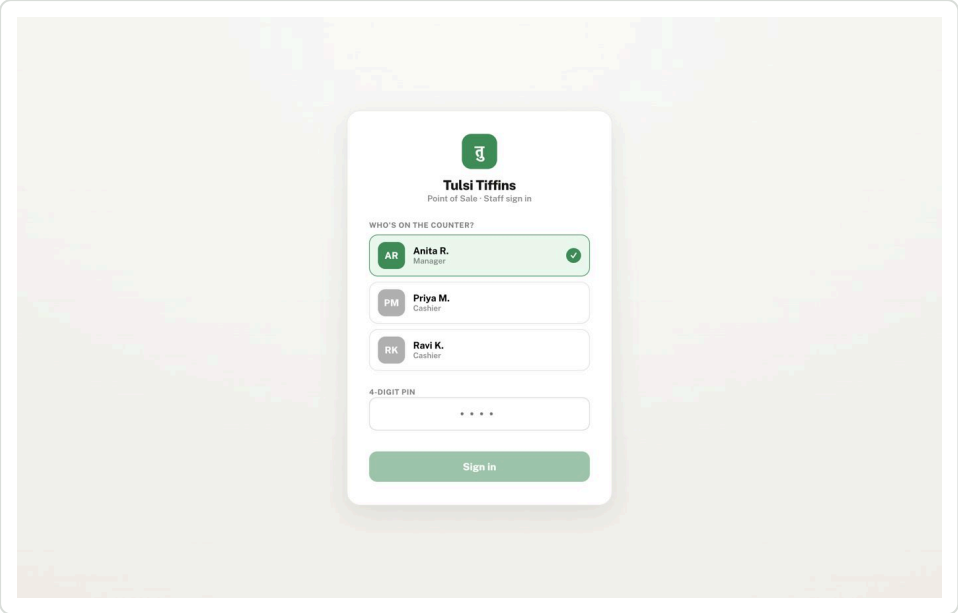


Figure 1: Login – tap a name, then enter the four-digit PIN screen capture

8.1 Sell (cashier or manager)

1. On the **Register**, filter by category if needed and tap a photo card to add the item to the order. Tap again or use + / − to change the quantity. A card cannot be added beyond its stock; sold-out items are greyed out.
2. The order panel shows subtotal, 5 % GST and total. Tap **Charge**.
3. **Cash:** enter the amount received; the change is shown, and *Payment received* becomes available once the amount covers the total. **UPI:** the customer scans the QR code; after checking their phone, tap *Mark paid*.
4. The receipt shows the bill number and can be printed. Stock is reduced at the same moment.

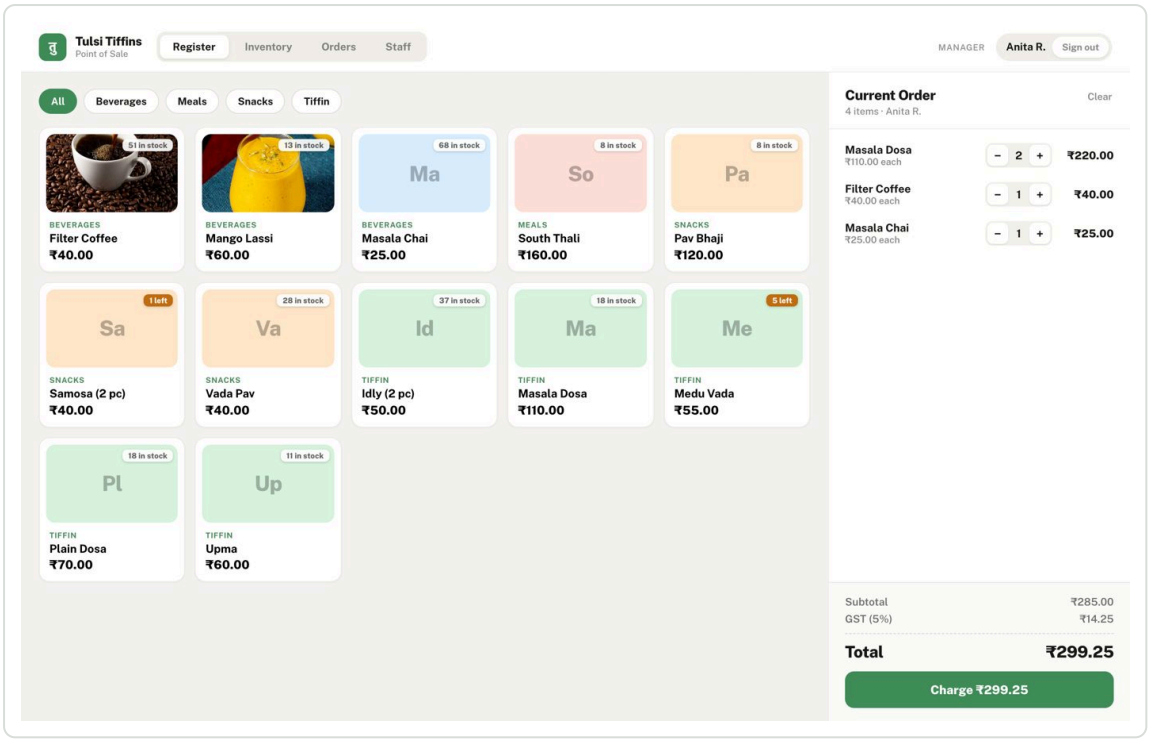


Figure 2: Register with photo cards, stock badges and the current order screen capture

8.2 Manage (manager only)

- **Inventory:** add or edit products (name, category, price, reorder level, photo); *Receive* or *Remove* stock – every change appears in the stock activity list with the user and time; archive products that are no longer sold.
- **Orders:** search bills by date or bill number, open a receipt, and see the day's total.
- **Staff:** add staff with name, role and PIN, reset a PIN, or deactivate an account. The last active manager cannot be deactivated.

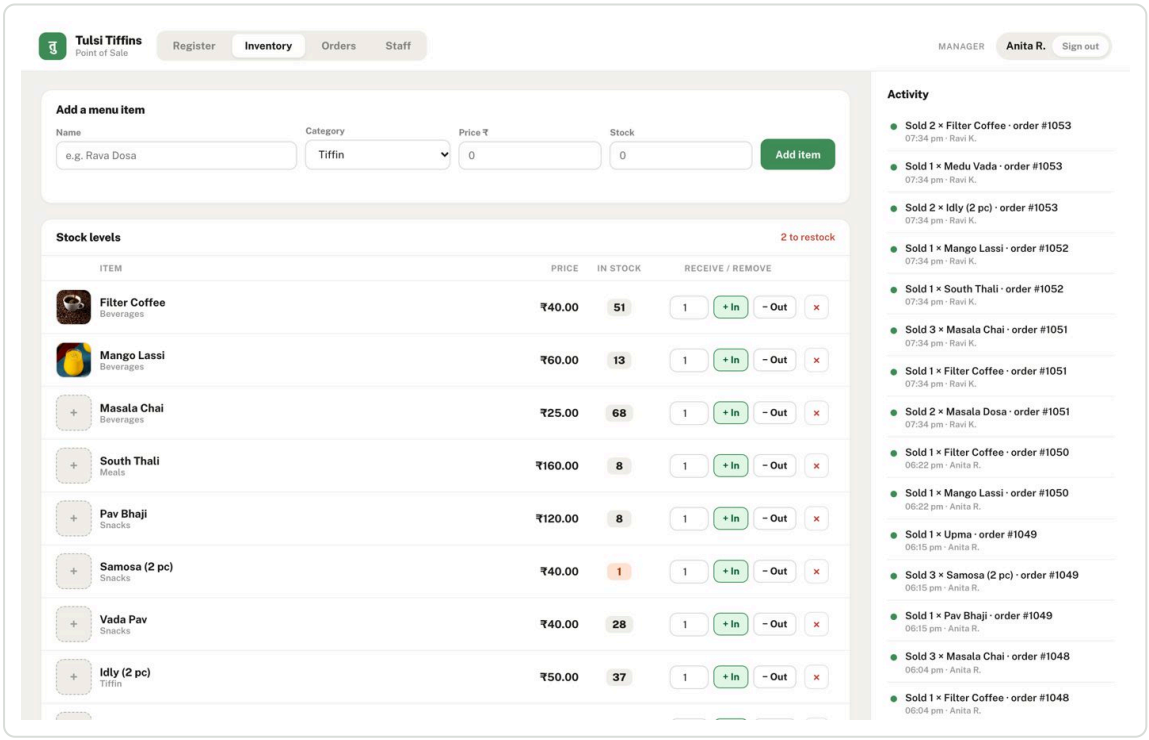


Figure 3: Inventory with stock levels and the stock activity ledger screen capture

Cashiers see only the Register and their bill history. The server checks the role of every request, so a cashier cannot reach manager functions even by calling the API directly (HTTP 403).

9 Troubleshooting

Symptom	Cause	Solution
Hosted app: login screen empty or loading for a long time	The free API is waking up	Wait 30–60 s and reload the page.
`Cannot connect to the Docker daemon`	Docker Desktop is not running	Start Docker Desktop and wait until it is ready.
`Bind for 0.0.0.0:5432 failed: port is already allocated`	Another database or container uses the port	Choose other ports (Section 3.5); `docker ps` shows what is running.
Browser shows "502 Bad Gateway" right after start	The API is still migrating	Wait a few seconds; check `docker compose logs api` for *starting uvicorn*.
"Incorrect PIN. Please try again."	Wrong account selected, or demo PIN was changed	Select the name first, then the PIN from Section 1.1. To restore the demo data locally, reset the database (Section 10).
"Insufficient stock: only N left"	The order asks for more than is in stock – expected behaviour	Reduce the quantity, or receive stock as a manager.
The app suddenly returns to the login screen	The 8-hour login has expired, or the account was deactivated	Log in again.
"Can't reach the server. Check the connection and try again."	Network down, API container stopped, or hosted API waking up	Check `docker compose ps` locally; for the hosted app wait a minute and retry.
All tests fail with `FileNotFoundError: alembic`	Virtual environment not on `PATH`	Use the exact command in Section 5.
All tests fail with `connection refused`	Database not running or on another port	`docker compose up -d db`; set `TEST_DATABASE_URL`.

10 Stopping, resetting and uninstalling

Goal	Command (in the repository folder)	Effect
Stop the application	`docker compose down` (or Ctrl + C)	Containers stop; all data is kept.
Start again	`docker compose up -d`	Starts in seconds with the existing data.
Update to a newer version	`git pull`, then `docker compose up -d --build`	Rebuilds images; new migrations are applied automatically; data is kept.
Back up the data	`docker compose exec db pg_dump -U tulsi tulsi_pos > backup.sql`	One SQL file with all sales, stock, staff and photos.
Reset to the demo state	`docker compose down -v`, then `docker compose up -d`	**Deletes all data**; the next start recreates the demo staff and menu.
Uninstall completely	`docker compose down -v --rmi local`, then delete the folder	Removes containers, the data volume and the built images.